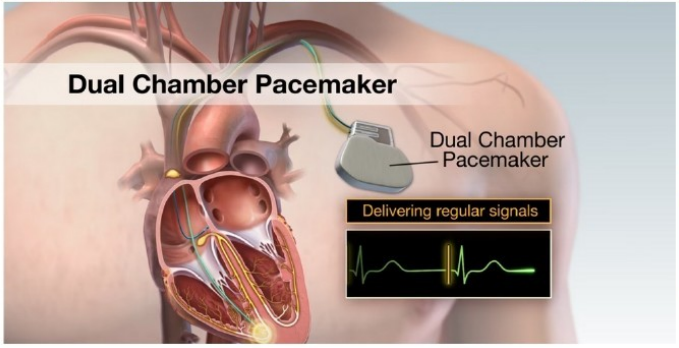
Ensuring Correctness at Runtime: Runtime Enforcement in Action

Ubiquitous Software and Software Systems



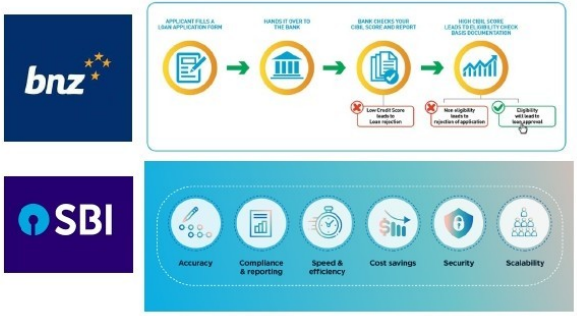


Figure: Source: Internet

Development Cycle

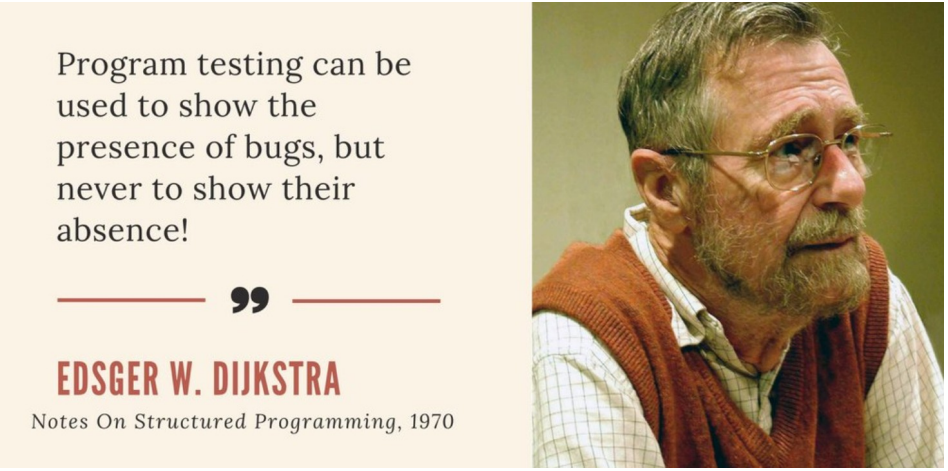
Traditional:

- Specification
- Design
- Implementation
- Testing
- Maintenance

Example: Software System (Banking Application)

- **Specification**: Define requirements (e.g., secure login, money transfer).
- **Design**: Plan architecture (modules for authentication, transaction).
- **Implementation**: Coding in Java/Python.
- **Testing**: Unit tests, integration tests.
- **Maintenance**: Bug fixes, updates.

Limitations of Testing



- Testing is labour intensive, hence expensive
- Exhaustive testing viable only for trivial systems.
- How to test for the unexpected? Rare cases?

Need of Verification

- Bugs caught early at design stage cost far less to fix than bugs detected after implementation/testing.
- Verification ensures correctness before coding, reducing cost and improving reliability.

Development Cycle (with Verification)

Traditional:

- Specification
- Design
- Verification
- Implementation
- Testing
- Maintenance

Example: Software System (Banking Application)

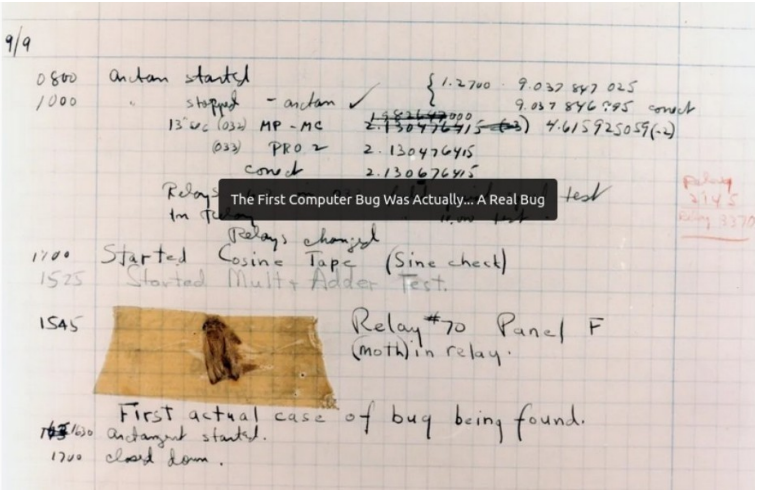
- **Specification**: Define requirements (e.g., secure login, money transfer).
- **Design**: Plan architecture (modules for authentication, transaction).
- **Verification:** Check correctness of design (e.g., login flow always leads to secure session).
- **Implementation**: Coding in Java/Python.
- **Testing**: Unit tests, integration tests.
- **Maintenance**: Bug fixes, updates.

Verification/Simulation

- Simulation: Performed on a model or design before implementation.
- **Goal**: verify correctness of the design early.
- Can be exhaustive/non-exhaustive depending on input coverage.

- **Exhaustive** Simulation:
  - Number of test cases grows exponentially with number of state variables.
  - Practically infeasible for complex systems.
- **Non-Exhaustive** Simulation:
  - Tests only a subset of possible inputs.
  - Finding the *right subset* is hard (**test-case generation problem**).
  - Bugs may still remain undiscovered.

Requirement of high-level confidence in behaviour



Pentium FDIV — software **bug** in HW: The floating point division algorithm uses an array of constants with 1066 elements. However, only 1061 elements of the array were correctly initialised.

Ariane 5: Exploded 37 seconds after takeoff - the reason was an overflow in a conversion of a 64 bit floating point number into a 16 bit integer.

Image © CPU-World.com

Motivation for Formal Verification

- Simulation increases confidence but not guaranteed correctness.
- Important to evaluate their behaviour/functionality.
  - ✓ product is defect-free, meets the specified requirements?
  - ✓ legislation and certification authorities require proof in terms of a documented verification process.
- **Formal Verification**
  - ➔ Uses mathematical reasoning to prove correctness of the design with respect to a specification.
  - ➔ Ensures system correctness *under all possible inputs and states*.
  - ➔ Detects design-level bugs early → saves cost, improves reliability.
  - ◆ **Methods**: model checking, theorem proving

Formal verification

We construct a model (for the application) in which we can demonstrate that certian property holds:

- system model
- specification (properties)

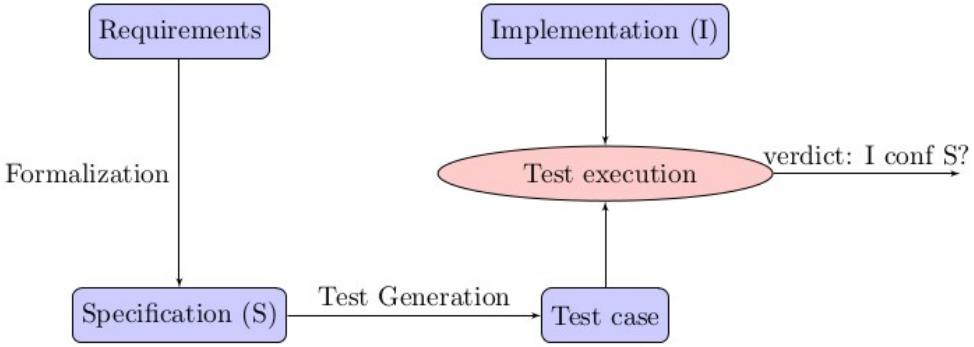
Formal verification techniques

- **Theorem proving** (proof in mathematics)
  - It is the process of using logical reasoning to show that a statement (theorem) follows from a given set of axioms and inference rules.
  - **Key idea**: You start with a formal specification of what you want to prove. You have a set of axioms (assumed truths) and rules of inference (ways to derive new truths). You apply these rules step-by-step until you either:
    - Derive the statement you want (proof complete), or
    - Find that it cannot be proven (counterexample or inconsistency).

- applied manually and demands a high degree of knowledge of the design under verification and of logic for the user to perform the verification
- complex formulas- difficult to automate

Formal verification techniques

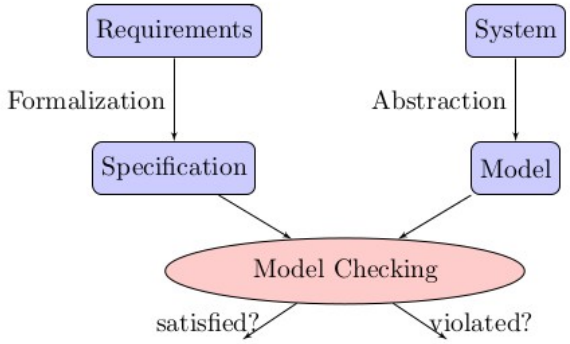
- **Conformance testing** (using formal methods)
  - Checking functional correctness, by means of testing, of a black-box system under test with respect to a formal system specification, i.e., a specification given in a language with a formal semantics.



Formal verification techniques

Model Checking

- Model Checking (MC) is an automated approach to verify that a formal model of the system satisfies the formal specification, defining the requirements of the system.



- Model
  - A model of the system is an abstraction of the system (omitting details irrelevant for checking the desired properties) that describes how the system may evolve over time.
  - Models are typically described as automata, describing the possible states, initial states, and the possible transitions between states.
- Specification
  - Specification are usually expressed in some form of temporal logic such as Linear-time Temporal Logic (LTL).
- Algorithm
  - The model checking algorithms (and tools), take the model and properties as input, and answer whether the model satisfies the properties (or not).
  - It explores all the possible states to decide whether the properties hold in all reachable states.
  - In case if a property does not hold, then it returns a counterexample (which is a run of the model of the system, leading to a state in which the property does not hold).
  - By studying the counterexample, one can pinpoint the source of the error in the model, correct the model, and try again.
  - The idea is that by ensuring that the model satisfies system properties, we increase our confidence in the correctness of the model.
- Model checkers
  - Spin and UPPAAL

Formal verification techniques

Model Checking

- Availability of system model: A suitable model of the system to be checked must be constructed—before actually running the system.
- State explosion problem: analyzing all executions of a system is done by generating the whole state space of the underlying system, which is often huge.
- Runtime uncertainities: Static verification techniques (like model checking)- might behave faulty at runtime, due to:
  - compiler bugs
  - mismatches of behaviour
  - runtime vulnerabilities

Formal verification techniques

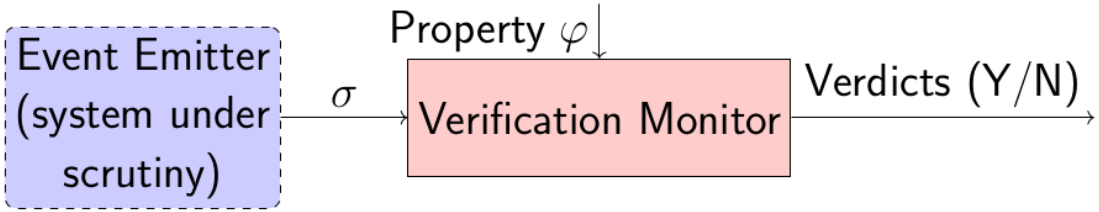
Runtime verification

- It is the discipline of computer science that deals with the study, development, and application of those verification techniques that allow checking whether a run of a system under scrutiny satisfies or violates a given correctness property (φ) .
- No formal model: It is being pursued as a lightweight verification technique complementing verification techniques such as model checking.
- One of the main distinguishing features of runtime verification is due to its nature of being performed at runtime, which opens up the possibility to act whenever incorrect behavior of a software system is detected.
- No repair, only detection: It itself deals (only) with the detection of violations (or satisfactions) of correctness properties. Thus, whenever a violation has been observed, it typically does not influence or change the program's execution, say for trying to repair the observed violation.

Formal verification techniques

Runtime verification

- Input (σ): system events (e.g., trace, log, messages)
- φ: Automata, temporal logic, etc. based formalism
- Does σ satisfy φ ?



- Checking whether an execution meets a correctness property is typically performed using a monitor.
- Monitors are typically generated automatically from some high-level specification.
- In its simplest form, a monitor decides whether the current execution satisfies a given correctness property by outputting either yes/true or no/false.

Formal verification techniques

Runtime verification

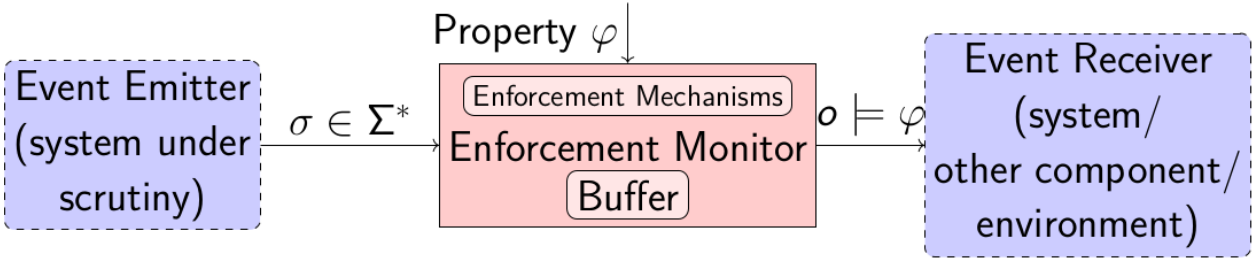
- Formally, when [[ϕ]] denotes the set of valid executions given by property ϕ, runtime verification boils down to checking whether the execution w is an element of [[ϕ]]. Thus, runtime verification answers the word problem, i.e. the problem whether a given word is included in some language.
- The complexity for generating the monitor is typically negligible, as the monitor is often only generated once.
- Online and offline monitoring: A monitor may on one hand be used to check the current execution of a system. In this setting, which is termed online monitoring, the monitor should be designed to consider executions in an incremental fashion. On the other hand, a monitor may work on a (finite set of) recorded execution(s), in which case we speak of offline monitoring.
- Properties of monitor: For a monitor to be ideally suited for runtime verification, it should adhere to the two maxims impartiality and anticipation.
  - Impartiality requires that a finite trace is not evaluated to true or, respectively false, if there still exists an (infinite) continuation leading to another verdict. Clearly, this maxim requires to have at least three different truth values: true, false, and inconclusive
  - Anticipation requires that once every (infinite) continuation of a finite trace leads to the same verdict, then the finite trace evaluates to this very same verdict. The second maxim requires a monitor to indeed report true or false, if the correctness property is indeed violated or satisfied.

Formal verification techniques

- Runtime enforcement
  - An alternative to passive RV- ensure that the (un-trustworthy) output satisfies desired properties
  - Enforcers are synthesized automatically, from a set of properties, which modify (untrustworthy) input into an output sequence that complies with the property.
  - The modifications are performed only in the event of a violation.
  - The modifications might include
    - blocking the execution,
    - modifying input sequence by suppressing and/or inserting actions,
    - buffering input actions until a future time when it could be forwarded.
      - In case of buffering, the enforcer is equipped with an internal memory (buffer) to store some events that are received as input which would be released as output only when the satisfaction of the property is ensured.

Formal verification techniques

- Runtime enforcement
  - The modifications performed by an EM should satisfy:
    - Soundness means that the resulting sequence obeys the desired specification property.
    - Transparency means that, the monitor should modify the input sequence in a minimal way. This means that when the enforcer checks the input sequence and finds that it already conforms to the property, then the input sequence should not be modified and should be simply released as output.
      - By adhering to the transparency constraint, the enforcer ensures faithfulness to the original behaviour of the system: if the system’s behaviour is correct (i.e., satisfies the property), the enforcer should not alter it.

Formal verification techniques

- Runtime enforcement
  - No formal model
  - Input (σ): system events
  - Modified to satisfy the property
  - Output (o): a sequence of events such that o |= φ